Pashov Audit Group

Nucleus Security Review

March 4th 2026 - March 5th 2026

Contents

1. About Pashov Audit Group 3
2. Disclaimer 3
3. Risk Classification 3
4. About Nucleus 4
5. Executive Summary 4
6. Findings 5

High findings 7

[H-01] Refund orders with frozen receivers can permanently halt withdrawals 7

Medium findings 8

[M-01] Deposit slippage can be bypassed by post-fee share deduction 8

Low findings 10

[L-01] `depositNative` checks `msg.value` before KYT unlike others 10
[L-02] `calculateOfferFees` can return fees greater than the principal amount 10
[L-03] `_lzReceive` validates hooks and pauses before allowlist 11
[L-04] Unused ETH in warp route flow is not refunded to users 12
[L-05] Supply cap enforcement uses raw share supply instead of value-based cap 13
[L-06] WarpRouteWrapper KYT attestation missing recipient check 13
[L-07] Cross-chain bridging burns shares for blacklisted recipients 14
[L-08] Frozen users can deposit assets that become impossible to withdraw 14
[L-09] Receivers on the freeze list can still withdraw through the queue 15
[L-10] WarpRouteWrapper allows minting bypassing fee and supply cap 15
[L-11] `BoringVault.fallback()` is not `payable` 16

1. About Pashov Audit Group

Pashov Audit Group consists of 40+ freelance security researchers, who are well proven in the space - most have earned over $100k in public contest rewards, are multi-time champions or have truly excelled in audits with us. We only work with proven and motivated talent.

With over 300 security audits completed — uncovering and helping patch thousands of vulnerabilities — the group strives to create the absolute very best audit journey possible. While 100% security is never possible to guarantee, we do guarantee you our team's best efforts for your project.

Check out our previous work here or reach out on Twitter @pashovkrum.

2. Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time, resource and expertise bound effort where we try to find as many vulnerabilities as possible. We can not guarantee 100% security after the review or even if the review will find any problems with your smart contracts. Subsequent security reviews, bug bounty programs and on-chain monitoring are strongly recommended.

3. Risk Classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users
- **Medium** - leads to a moderate material loss of assets in the protocol or moderately harms a group of users
- **Low** - leads to a minor material loss of assets in the protocol or harms a small group of users

Likelihood

- **High** - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost
- **Medium** - only a conditionally incentivized attack vector, but still relatively likely
- **Low** - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive

4. About Nucleus

Nucleus is a vault management platform built on the BoringVault architecture that enables cross-chain asset operations, multi-asset deposits, and configurable fee routing. The Rushmore upgrade introduces changes to cross-chain tellers, asset-specific fee modules, deposit distribution via code depositors, Hyperlane warp route integration, and enhanced decoder sanitization.

5. Executive Summary

A time-boxed security review of the **paxoslabs/nucleus-boring-vault** repository was done by Pashov Audit Group, during which **0xAlix2**, **BenRai**, **BengalCatBalu** engaged to review **Nucleus**. A total of **13** issues were uncovered.

Protocol Summary	
Project Name	Nucleus
Protocol Type	Vault Management
Timeline	March 4th 2026 - March 5th 2026

Review commit hash:

- 4d462c0c24f787c398fa6432d68955b5c9462398 (paxoslabs/nucleus-boring-vault)

Fixes review commit hash:

- f8cec32e4ad5fae957f8f280361d4f1ec416d2b5 (paxoslabs/nucleus-boring-vault)

Scope

`BoringVault.sol` `NucleusDecoderAndSanitizer.sol` `CrossChainTellerBase.sol` `MultiChainHyperlaneTellerWithMultiAssetSupport.sol` `MultiChainLayerZeroTellerWithMultiAssetSupport.sol` `TellerWithMultiAssetSupport.sol` `TellerWithMultiAssetSupportPredicateProxy.sol` `AssetSpecificFeeModule.sol` `DexAggregatorWrapperWithPredicateProxy.sol` `DistributorCodeDepositor.sol` `FreezeListBeforeTransferHook.sol` `WarpRouteWrapper.sol` `BeforeTransferHook.sol` `DecoderCustomTypes.sol` `IFallbackHook.sol`

6. Findings

Findings count

Severity	Amount
High	1
Medium	1
Low	11
Total findings	**13**

Summary of findings

ID	Title	Severity	Status
[H-01]	Refund orders with frozen receivers can permanently halt withdrawals	High	Resolved
[M-01]	Deposit slippage can be bypassed by post-fee share deduction	Medium	Resolved
[L-01]	`depositNative` checks `msg.value` before KYT unlike others	Low	Resolved
[L-02]	`calculateOfferFees` can return fees greater than the principal amount	Low	Resolved
[L-03]	`_lzReceive` validates hooks and pauses before allowlist	Low	Resolved
[L-04]	Unused ETH in warp route flow is not refunded to users	Low	Resolved
[L-05]	Supply cap enforcement uses raw share supply instead of value-based cap	Low	Resolved
[L-06]	WarpRouteWrapper KYT attestation missing recipient check	Low	Resolved
[L-07]	Cross-chain bridging burns shares for blacklisted recipients	Low	Acknowledged
[L-08]	Frozen users can deposit assets that become impossible to withdraw	Low	Acknowledged
[L-09]	Receivers on the freeze list can still withdraw through the queue	Low	Resolved

ID	Title	Severity	Status
[L-10]	WarpRouteWrapper allows minting bypassing fee and supply cap	Low	Resolved
[L-11]	`BoringVault.fallback()` is not `payable`	Low	Resolved

High findings

[H-01] Refund orders with frozen receivers can permanently halt withdrawals

Severity

Impact: Medium

Likelihood: High

Description

The `WithdrawQueue.processOrders()` function processes refund orders by burning the user's receipt token and calling `_refundOrder()` to transfer shares back to the `refundReceiver` without any failure handling on the `REFUND` path. If the `refundReceiver` address is on the `FreezeListBeforeTransferHook` freeze list configured on the underlying `BoringVault`, the share transfer reverts inside `_refundOrder()`. Because `processOrders()` advances `lastProcessedOrder` sequentially and does not provide a mechanism to skip or quarantine failing `REFUND` orders, the queue becomes stuck at the first such order and cannot progress to subsequent entries. A malicious user can deliberately craft this condition by submitting a withdrawal, setting a frozen `refundReceiver`, cancelling to mark the order as `REFUND`, and permanently denying Service (DoS) withdrawals for all later users.

Recommendation

Introduce a resilient refund path for `REFUND` orders that cannot successfully transfer shares to `refundReceiver`. One option is to detect vault transfer failures (e.g., via `try/catch` or by checking freeze status through the hook) and instead burn the shares or redirect them to a controlled fallback recipient after emitting an event for off-chain accounting.

Medium findings

[M-01] Deposit slippage can be bypassed by post-fee share deduction

Severity

Impact: Medium

Likelihood: Medium

Description

`DistributorCodeDepositor._deposit()` relies on `teller.deposit()` to enforce the `minimumMint` slippage bound, but then applies deposit fees to the gross minted shares before transferring them to the user.

```
function _deposit(
        ERC20 depositAsset,
        uint256 depositAmount,
        uint256 minimumMint,
        address to,
        bytes calldata distributorCode
){  ...
        shares = teller.deposit(depositAsset, depositAmount, minimumMint);

        uint256 feeAmount;

        // if fee module is zero, no fees
        if (address(feeModule) != address(0)) {
            feeAmount = feeModule.calculateOfferFees(shares, IERC20(address(depositAsset)),
IERC20(boringVault), to);

        if (feeAmount >= shares) revert FeesExceedOrEqualShares();

        // Send the fees to the fee recipient if fees are taken
        if (feeAmount > 0) {
        ERC20(boringVault).safeTransfer(feeRecipient, feeAmount);

        }
        // Send "to" the shares - fees
        ERC20(boringVault).safeTransfer(to, shares - feeAmount);

...
}
```

Because `minimumMint` is only checked on the pre-fee share amount, an attacker or adverse market move can cause the user to receive `shares - feeAmount` strictly below their stated `minimumMint` without reverting. This breaks the expected slippage guarantee for depositors and enables sandwich-style attacks where a user is induced to accept fewer vault shares than they explicitly require.

Recommendation

After computing `feeAmount` and before transferring shares to the user, recompute an effective post-fee share amount and enforce `shares - feeAmount >= minimumMint`. If the inequality does not hold, revert the transaction instead of transferring shares. This ensures the slippage constraint applies to the actual number of shares that the user receives after protocol fees.

Low findings

[L-01] `depositNative` checks `msg.value` before KYT unlike others

Description

In `src/helper/DistributorCodeDepositor.sol`, `deposit()` and `depositWithPermit()` both call `_authorizeKyt()` before any fund validation or transfer. `depositNative()` reverses this; it validates `msg.value != depositAmount` at line 179 before calling `_authorizeKyt()`.

This inconsistency means a user who sends the wrong `msg.value` and would also fail KYT gets `IncorrectNativeDepositAmount` instead of `UnauthorizedTransaction`, unlike the other two entry points, which surface the KYT error first.

Recommendations:

Consider moving `_authorizeKyt()` before the `msg.value` check in `depositNative()` to match the authorization-first pattern of the other two entry points.

[L-02] `calculateOfferFees` can return fees greater than the principal amount

Description

In `src/helper/AssetSpecificFeeModule.sol`, `calculateOfferFees()` computes the total fee as `percentageFee + flatFee` with no check that the result is less than or equal to the input `amount`:

```
// AssetSpecificFeeModule.sol, lines 50-58
function calculateOfferFees(
    uint256 amount,
    IERC20 offerAsset,
    IERC20 wantAsset,
    address receiver
) external view override returns (uint256 feeAmount) {
    FeeData memory feeData = depositTokenFeeData[offerAsset];
    uint256 percentageFee = amount.mulDivUp(feeData.feePercentage, ONE_HUNDRED_PERCENT);
    uint256 flatFee = feeData.flatFee;

    feeAmount = percentageFee + flatFee;
}
```

When `flatFee` is non-trivial relative to the deposit size (e.g., a small deposit against a flat fee denominated in the asset's base units), `feeAmount` can exceed `amount`. The current primary consumer, `DistributorCodeDepositor._deposit()`, has its own guard (`if (feeAmount >= shares) revert FeesExceedOrEqualShares()`), so the DCD is protected.

However, the fee module is designed as a standalone, reusable component. The developer documentation explicitly states that it "may possibly be used to apply fees per asset for the withdrawal queues as well." Any future consumer that calls `calculateOfferFees()` and subtracts the fee from the principal without its own bounds check would face an underflow revert (or, in an unchecked context, a wrapping error).

`WithdrawQueue.processOrders()` is a confirmed affected consumer: it calls `calculateOfferFees()` without a guard, so when `feeAmount > order.amountOffer`, the subtraction `order.amountOffer - feeAmount` underflows and reverts. Because `processOrders()` advances `lastProcessedOrder` sequentially without per-order fallback, the entire withdrawal queue is permanently halted at the first such order until an admin manually intervenes.

Recommendations:

Consider documenting this behavior, so that any consumers are aware of this. In consumers that subtract the fee from the principal (e.g., `WithdrawQueue.processOrders()`), add an explicit guard before the subtraction: `if (feeAmount > order.amountOffer) { /* skip or refund the order */ }` so that an oversized fee triggers a per-order fallback path rather than halting the entire queue.

[L-03] `_lzReceive` validates hooks and pauses before allowlist

Description

In `src/base/Roles/CrossChain/MultiChainLayerZeroTellerWithMultiAssetSupport.sol`, `_lzReceive()` performs operations in the following order:

1. Decode payload → `(shareAmount, receiver)`
2. Call `_beforeReceive(shareAmount, receiver)`, which checks `isPaused` and calls `vault.hook().beforeReceiveBridge(shareAmount, receiver)` (`CrossChainTellerBase.sol`)
3. Check `selectorToChains[_origin.srcEid].allowMessagesFrom`
4. Mint via `vault.enter(...)`

The `_beforeReceive` call can revert in two ways before the origin check is ever reached:

- `TellerWithMultiAssetSupport__Paused()` if `isPaused` is true (`CrossChainTellerBase.sol`)
- `FrozenAddress(receiver)` if `receiver` is on the freeze list (`FreezeListBeforeTransferHook.beforeReceiveBridge`)

This means a message from a **disallowed chain** can revert with `Paused` or `FrozenAddress` instead of the intended `MultiChainTellerBase_MessagesNotAllowedFrom`, masking the real reason for rejection.

The Hyperlane implementation in `handle()` (`MultiChainHyperlaneTellerWithMultiAssetSupport.sol`) intentionally validates origin/sender/allowlist **before** decoding and running hooks.

This creates inconsistencies with surfacing errors.

Recommendations:

Consider reordering validations to ensure that the origin is checked first.

[L-04] Unused ETH in warp route flow is not refunded to users

Description

The `WarpRouteWrapper` flow accepts `msg.value` from users to perform a sequence of operations but does not ensure that any unused ETH is returned once the route is complete. If the user sends too much ETH with the transaction, the surplus remains trapped in the contract instead of being refunded to the caller, even if it is refunded to the contract. WarpRouteWrapper is Auth, PredicateClient { //NEW - PredicateClient was added. This breaks the expectation that the user receives unused native tokens back.

Recommendation

Ensure that any ETH not consumed by the route is explicitly refunded to the original caller at the end of the function. After finishing all internal calls that may use `msg.value`, compute the remaining ETH balance attributable to the current operation and transfer it back to `msg.sender`. For example, track the contract's ETH balance before the operation, then after all steps, refund `address(this).balance - balanceBefore` to `msg.sender` (guarding against reentrancy and ensuring the refund happens after all external calls or using a pull pattern if necessary). This guarantees that the helper only keeps ETH strictly required for operations and that users never lose excess ETH due to wrapper over-collection.

[L-05] Supply cap enforcement uses raw share supply instead of value-based cap

Description

`DistributorCodeDepositor` enforces a `supplyCap` by comparing `ERC20(boringVault).totalSupply()` directly against the cap, implicitly assuming that share supply is a good proxy for value under management. In practice, the underlying vault value depends on the asset mix and oracle rates used by the accountant, so a fixed share cap can either be materially below or above the intended value cap over time. This undermines the protocol's stated goal of preventing deposits that exceed available liquidity in reference markets and can allow either excessive risk concentration or unnecessary denial of deposits during rate shifts.

Recommendation

Replace the share-only cap with a value-based limit derived from the accountant's pricing, e.g., by computing the total vault value in a reference denomination and comparing that amount against a configurable cap.

[L-06] WarpRouteWrapper KYT attestation missing recipient check

Description

In `src/helper/WarpRouteWrapper.sol`, the Predicate KYT attestation encodes:

```
bytes memory encodedSigAndArgs =
    abi.encodeWithSignature("deposit(address,uint256,uint256)", depositAsset, depositAmount,
minimumMint);
```

The `recipient` (`bytes32`, the bridge destination address) is not included in `encodedSigAndArgs`. Since the Predicate API generates attestations based on the transaction context it receives, and the on-chain registry verifies the attestation against exactly this encoded payload, the compliance policy structurally cannot screen the bridge destination, even if configured to do so.

This contradicts how `PredicateClient` is intended to be used. Per the [Predicate smart contract integration docs](), `PredicateClient` exists specifically for when "your policy validates specific function calls, parameters, or value-based limits." The documented pattern is to encode the actual protected function with all its arguments:

```
bytes memory encodedSigAndArgs = abi.encodeWithSignature(
    "_deposit(uint256)",
    _amount
);
```

The DCD correctly follows this pattern at `src/helper/DistributorCodeDepositor.sol`, encoding `_deposit(address,uint256,uint256,address,bytes)`, which includes the recipient (`to`).

Additionally, the encoded signature `"deposit(address,uint256,uint256)"` refers to the teller's `deposit` function, not the wrapper's actual entry point `depositAndBridge`. This means the attestation does not bind to the operation actually being executed (a deposit plus bridge), further weakening the semantic integrity of the KYT proof.

The compliance policy cannot distinguish between deposit-only and deposit-and-bridge authorization at the on-chain verification boundary.

Recommendations

Consider encoding the actual `depositAndBridge` function signature with all security-relevant parameters, including the `recipient`.

[L-07] Cross-chain bridging burns shares for blacklisted recipients

Description

If a user calls CrossChainTellerBase.bridge(), it is only checked if the msg.sender is on the freeze list but not the recipient. Because freeze lists or blacklist configurations can differ across chains, it is possible for a user to specify a destination recipient that is frozen on the destination chain even if the sender is not blacklisted on the source chain. In that case, shares are burned on the source, the destination mint reverts or is blocked, and the user's position is effectively lost.

Recommendation

Align freeze-list semantics across chains used by the protocol, and perform an explicit pre-bridge check that the intended `destinationChainReceiver` is acceptable according to the destination chain's policy. At a minimum, introduce configurable allowlists or registry-based checks that prevent bridging to known-blocked recipients, or route such attempts into an escrow-like mechanism where the shares can be reclaimed or redirected rather than burned irreversibly.

[L-08] Frozen users can deposit assets that become impossible to withdraw

Description

`TellerWithMultiAssetSupport.deposit()` allows a user to deposit assets and mint vault shares without consulting the `BeforeTransferHook` that enforces the freeze list on the `BoringVault`. If a user is (unknowingly) frozen at the vault level, they can still acquire new

shares via this deposit path. These shares cannot later be transferred out of the user's address (including to a withdrawal queue), creating stuck positions and inconsistent enforcement between deposit and transfer.

Recommendation

Before minting or assigning new vault shares, mirror the same freeze checks that would be executed on a transfer to or from the depositor to ensure that a frozen user cannot deposit into the vault.

[L-09] Receivers on the freeze list can still withdraw through the queue

Description

In `WithdrawQueue.processOrders()`, the protocol checks that the receiver is the current owner of the receipt token but does not consult the `BeforeTransferHook` freeze list before executing the withdrawal. As a result, an account that will be frozen at the vault level can still front-run the freeze operation and successfully process a queued withdrawal, moving value out of the system even though their shares should be immobilized. This weakens the security guarantees of the freeze mechanism and allows circumvention of governance or compliance decisions that rely on blocking transfers for certain addresses.

Recommendation

Before burning the receipt token and calling `bulkWithdraw()`, invoke the same freeze checks used for share transfers (e.g. through the vault's `BeforeTransferHook`) against the `receiver`. If the receiver is on the freeze list, do not process the withdrawal; instead, burn or quarantine the associated shares and emit an event so that governance can decide on further action. This ensures that queued withdrawals respect the same freezing rules as direct share transfers.

[L-10] WarpRouteWrapper allows minting bypassing fee and supply cap

Description

In `src/helper/DistributorCodeDepositor.sol`, `_deposit()` applies (optionally) feeModule-based share fees and enforces `totalSupply <= supplyCap` after minting, reverting if exceeded. In contrast, `src/helper/WarpRouteWrapper.sol:depositAndBridge()` is publicly callable (no requiresAuth) and mints shares by calling `teller.deposit(depositAsset, depositAmount, minimumMint)` but does not apply any fee module logic and does not enforce any supply cap check.

Since `teller.deposit()` is `requiresAuth`-gated, the `WarpRouteWrapper` must be granted teller authorization in production; once authorized, any external caller can use `depositAndBridge()` to mint shares without the DCD's fee and cap restrictions.

Recommendations:

If fees/caps are intended to apply to all share issuance through wrappers, consider adding the same fee deduction and supply cap enforcement to `WarpRouteWrapper`. If `WarpRouteWrapper` is intentionally fee/cap-free, consider explicitly documenting that it bypasses DCD controls.

[L-11] `BoringVault.fallback()` is not `payable`

Description

The protocol documentation describes the fallback hook as a pseudo-upgradeability mechanism that allows adding new functionality to the vault without a full proxy upgrade: *"This hook may choose to implement getter functions or anything else we may want to add to the BoringVault."* The design explicitly intends to keep the upgrade surface low risk by restricting hooks to operations that do not touch BoringVault storage.

`BoringVault` defines a `receive()` for plain ETH and a `fallback()` for calldata-bearing calls:

```
receive() external payable { }  // line 194

fallback(bytes calldata data) external returns (bytes memory result) {  // line 202 — not
payable
    if (address(fallbackHook) == address(0)) revert();
    result = fallbackHook.onFallback(msg.sender, data);
}
```

Because `fallback` is not marked `payable`, any call with non-empty calldata and a non-zero `msg.value` reverts at the EVM level before `onFallback` is reached. This silently contradicts the documented intent of "anything else we may want to add": if a future hook needs to implement a native ETH deposit getter, a price query with a commitment, or any operation where the caller attaches ETH alongside custom calldata, it is structurally blocked by the non-payable fallback — without any error that points to this constraint.

Recommendation:
Document in the `fallback` NatSpec and in `IFallbackHook` that ETH cannot be forwarded through the fallback. If future hooks may need to handle ETH, mark `fallback` as `payable` and forward `msg.value` to `onFallback`.